

Année 2025–2026 û L3 MIAGE û Graphe & Open Data

# RAPPORT TECHNIQUE

## Scraping, ETL Assisté par IA, et Analyse de Graphes Episteme Network

---

|                     |                                              |
|---------------------|----------------------------------------------|
| <b>Projet</b>       | Episteme Network                             |
| <b>Cours</b>        | Graphe et Open Data                          |
| <b>Formation</b>    | L3 MIAGE - Université Paris Nanterre         |
| <b>Auteur</b>       | Terrel NUENTSA                               |
| <b>Technologies</b> | Python, NetworkX, LLM (Groq/Mistral), Vis.js |
| <b>Contact</b>      | nuentsa.terrel@gmail.com                     |

*Ce rapport présente l'architecture technique d'un système de construction automatique de graphes de connaissances représentant les influences intellectuelles entre scientifiques à travers l'histoire.*

# Table des matières

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction et Contexte</b>                     | <b>4</b>  |
| 1.1      | Présentation du Projet . . . . .                    | 4         |
| 1.2      | Problématique Scientifique . . . . .                | 4         |
| 1.3      | Objectifs Pédagogiques et Techniques . . . . .      | 5         |
| 1.4      | Glossaire Technique . . . . .                       | 5         |
| <b>2</b> | <b>Architecture Globale du Système</b>              | <b>6</b>  |
| 2.1      | Vue d'Ensemble . . . . .                            | 6         |
| 2.2      | Organisation des Modules . . . . .                  | 6         |
| 2.3      | Flux de Données . . . . .                           | 7         |
| <b>3</b> | <b>Stratégie Open Data et Scraping</b>              | <b>8</b>  |
| 3.1      | Source de Données : Wikipédia . . . . .             | 8         |
| 3.2      | Fonctionnalités du Module WikipediaClient . . . . . | 8         |
| 3.2.1    | Recherche Floue (Fuzzy Search) . . . . .            | 8         |
| 3.2.2    | Détection Automatique des Scientifiques . . . . .   | 9         |
| 3.2.3    | Extraction des Métadonnées Temporelles . . . . .    | 9         |
| 3.2.4    | Classification par Domaine Scientifique . . . . .   | 9         |
| 3.3      | Gestion des Contraintes Techniques . . . . .        | 9         |
| 3.3.1    | Rate Limiting et Éthique du Scraping . . . . .      | 9         |
| 3.3.2    | Optimisation de l'Utilisation des Tokens . . . . .  | 10        |
| <b>4</b> | <b>Extraction d'Information par LLM</b>             | <b>11</b> |
| 4.1      | Stratégie Multi-Fournisseurs . . . . .              | 11        |
| 4.2      | Prompt Engineering . . . . .                        | 11        |
| 4.2.1    | Analyse des Choix de Conception du Prompt . . . . . | 12        |
| 4.2.2    | Paramètres d'Inférence . . . . .                    | 12        |
| 4.3      | Parsing Robuste des Réponses . . . . .              | 12        |
| <b>5</b> | <b>Système de Cache Intelligent</b>                 | <b>14</b> |
| 5.1      | Motivation . . . . .                                | 14        |
| 5.2      | Architecture du Cache . . . . .                     | 14        |
| 5.2.1    | Génération de Clé MD5 . . . . .                     | 14        |
| 5.2.2    | Versioning du Prompt . . . . .                      | 14        |
| 5.3      | Performances Observées . . . . .                    | 15        |
| <b>6</b> | <b>Construction et Validation du Graphe</b>         | <b>16</b> |
| 6.1      | Algorithme BFS de Construction . . . . .            | 16        |
| 6.1.1    | Paramètres de Configuration . . . . .               | 17        |
| 6.2      | Validation Chronologique . . . . .                  | 17        |

|           |                                                                |           |
|-----------|----------------------------------------------------------------|-----------|
| 6.2.1     | Exemples de Validation . . . . .                               | 18        |
| 6.3       | Système de Filtrage . . . . .                                  | 18        |
| 6.3.1     | Liste Noire (Blacklist) . . . . .                              | 18        |
| 6.3.2     | Patterns d'Exclusion (Regex) . . . . .                         | 18        |
| 6.4       | Validation Wikidata (Cross-Référence) . . . . .                | 19        |
| <b>7</b>  | <b>Algorithmes d'Analyse de Graphe</b>                         | <b>20</b> |
| 7.1       | Modèle Mathématique . . . . .                                  | 20        |
| 7.1.1     | Définition Formelle . . . . .                                  | 20        |
| 7.1.2     | Propriétés du Graphe . . . . .                                 | 20        |
| 7.2       | Métriques de Centralité . . . . .                              | 20        |
| 7.2.1     | Centralité de Degré (Degree Centrality) . . . . .              | 20        |
| 7.2.2     | PageRank . . . . .                                             | 21        |
| 7.2.3     | Centralité d'Intermédierité (Betweenness Centrality) . . . . . | 21        |
| 7.3       | Détection de Communautés : Algorithme de Louvain . . . . .     | 21        |
| 7.4       | Analyse des Trous Structurels (Paradigm Shifters) . . . . .    | 22        |
| 7.4.1     | Contrainte de Burt . . . . .                                   | 22        |
| 7.4.2     | Score Composite de Révolutionnaire . . . . .                   | 22        |
| 7.5       | Prédiction de Liens Manquants . . . . .                        | 22        |
| 7.5.1     | Similarité de Jaccard . . . . .                                | 22        |
| 7.5.2     | Indice d'Adamic-Adar . . . . .                                 | 23        |
| 7.6       | Analyse Topologique Globale . . . . .                          | 23        |
| 7.6.1     | Effet "Small-World" (Watts-Strogatz) . . . . .                 | 23        |
| 7.6.2     | Distribution "Scale-Free" (Barabási-Albert) . . . . .          | 23        |
| <b>8</b>  | <b>Visualisation Interactive</b>                               | <b>24</b> |
| 8.1       | Architecture Frontend . . . . .                                | 24        |
| 8.2       | Algorithme de Layout : Barnes-Hut . . . . .                    | 24        |
| 8.3       | Encodage Visuel . . . . .                                      | 25        |
| 8.3.1     | Taille des Nuds . . . . .                                      | 25        |
| 8.3.2     | Couleur des Nuds . . . . .                                     | 25        |
| 8.4       | Fonctionnalités Interactives . . . . .                         | 25        |
| <b>9</b>  | <b>Tests et Assurance Qualité</b>                              | <b>26</b> |
| 9.1       | Stratégie de Tests . . . . .                                   | 26        |
| 9.1.1     | Organisation des Tests . . . . .                               | 26        |
| 9.1.2     | Cas de Test Clés . . . . .                                     | 26        |
| 9.2       | Métriques de Qualité . . . . .                                 | 27        |
| <b>10</b> | <b>Conclusion et Perspectives</b>                              | <b>28</b> |
| 10.1      | Bilan Technique . . . . .                                      | 28        |
| 10.2      | Résultats Quantitatifs . . . . .                               | 29        |
| 10.3      | Limitations et Améliorations Futures . . . . .                 | 29        |
| 10.3.1    | Limitations Actuelles . . . . .                                | 29        |
| 10.3.2    | Perspectives d'Évolution . . . . .                             | 29        |
| 10.4      | Compétences Mobilisées . . . . .                               | 29        |
| <b>A</b>  | <b>Galerie Visuelle</b>                                        | <b>31</b> |
| A.1       | Interface Principale . . . . .                                 | 31        |

---

|          |                                             |           |
|----------|---------------------------------------------|-----------|
| A.2      | Détail d'un Scientifique . . . . .          | 32        |
| A.3      | Visualisation des Communautés . . . . .     | 32        |
| <b>B</b> | <b>Annexes Visuelles</b>                    | <b>33</b> |
| B.1      | Interface Principale (Vue Graphe) . . . . . | 33        |
| B.2      | Rapport d'Analyse (Small World) . . . . .   | 34        |

# Chapitre 1

## Introduction et Contexte

### 1.1 Présentation du Projet

**Episteme Network** est un système de *Data Science* appliqué à l'histoire des sciences. Il construit automatiquement un graphe orienté représentant les relations d'influence intellectuelle entre scientifiques à travers les siècles, en transformant des données textuelles non structurées (biographies Wikipédia) en connaissances structurées exploitables.

#### Chiffres Clés du Projet

- **1 726 scientifiques** analysés et intégrés au graphe
- **2 520 relations** d'influence extraites et validées
- **901 ans** d'histoire couverts (1105 - 2006)
- **10 domaines scientifiques** représentés
- **1 978 requêtes LLM** mises en cache

### 1.2 Problématique Scientifique

L'histoire des sciences forme un système complexe où les idées se transmettent, se transforment et s'influencent mutuellement. Reconstruire ce réseau d'influences pose plusieurs défis majeurs :

1. **Volume et Hétérogénéité** : Des centaines de milliers de scientifiques ont contribué au savoir humain. Leurs biographies varient considérablement en qualité et en structure.
2. **Non-structuration des Données** : Les relations d'influence n'existent pas sous forme de base de données. Elles sont enfouies dans du texte libre, exprimées de manières très diverses : *í élève de ž*, *í influencé par ž*, *í a travaillé avec ž*, *í a développé les idées de ž*, etc.
3. **Complexité Sémantique** : Il faut distinguer collaboration (A et B travaillent ensemble) d'influence (A a utilisé les travaux de B), et différencier les relations bidirectionnelles des relations asymétriques.
4. **Validation Temporelle** : Un système automatique peut produire des anachronismes (Newton influencé par Einstein). Une validation chronologique rigoureuse est indispensable.

**Question de recherche :** Comment automatiser la reconstruction de ce réseau socio-épistémique avec une précision suffisante pour révéler des structures cachées dans l’histoire des idées ?

## 1.3 Objectifs Pédagogiques et Techniques

Ce projet démontre la maîtrise de trois compétences fondamentales du cours :

| Compétence                      | Application dans le projet                                                                                                                |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Open Data &amp; Scraping</b> | Exploitation de l’API Wikipédia, extraction structurée, respect des contraintes éthiques (rate limiting, User-Agent)                      |
| <b>IA &amp; NLP</b>             | Utilisation de LLM (Llama 3.3, Mistral) pour l’extraction d’entités et de relations depuis du texte bruité                                |
| <b>Théorie des Graphes</b>      | Application d’algorithmes de centralité (PageRank, Betweenness), détection de communautés (Louvain), analyse des trous structurels (Burt) |

## 1.4 Glossaire Technique

### BFS (Breadth-First Search)

Algorithme de parcours en largeur d’abord. Explore tous les nuds à distance  $n$  avant ceux à distance  $n + 1$ .

### ETL (Extract, Transform, Load)

Pipeline de traitement de données en trois phases : extraction, transformation, chargement.

### GEXF

Format XML standard pour l’échange de graphes, compatible avec Gephi.

### Hallucination (LLM)

Phénomène où un modèle de langage génère des informations plausibles mais fausses.

### LLM (Large Language Model)

Modèle de langage de grande taille capable de comprendre et générer du texte.

### Louvain

Algorithme de détection de communautés par optimisation de la modularité.

### PageRank

Algorithme de Google mesurant l’importance d’un nud selon l’importance de ses voisins entrants.

### SPARQL

Langage de requête pour interroger des bases de données RDF, notamment Wikidata.

### Structural Holes

Théorie de Burt (1992) : les individus connectant des groupes séparés ont un avantage stratégique.

# Chapitre 2

## Architecture Globale du Système

### 2.1 Vue d'Ensemble

Le système Episteme Network est organisé en **pipeline ETL moderne assisté par IA**. Chaque module a une responsabilité unique (principe SRP - Single Responsibility Principle) et communique via des interfaces clairement définies.

### 2.2 Organisation des Modules

Le projet est structuré en **8 modules Python principaux**, chacun avec une responsabilité unique :

| Module              | Responsabilité                                                                                                                   |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------|
| wikipedia_client.py | Interaction avec l'API Wikipédia : recherche floue, extraction de texte, détection de domaine scientifique, extraction des dates |
| llm_extractor.py    | Communication avec les LLM (5 fournisseurs supportés), prompt engineering, parsing des réponses JSON                             |
| cache_manager.py    | Système de cache basé sur MD5 avec versioning du prompt pour invalidation automatique                                            |
| graph_builder.py    | Algorithme BFS de construction du graphe, validation chronologique, filtrage des noms                                            |
| validator.py        | Cross-référencement avec Wikidata via SPARQL pour validation des relations                                                       |
| visualizer.py       | Génération de l'interface HTML interactive avec Vis.js                                                                           |
| config.py           | Configuration centralisée : clés API, paramètres, blacklist, patterns d'exclusion                                                |
| logger.py           | Utilitaires de journalisation                                                                                                    |

TABLE 2.1 – Modules principaux et leurs responsabilités

En complément, le dossier **scripts/** contient **30+ scripts d'analyse** : calcul de métriques (PageRank, Betweenness), détection de communautés, identification des paradigm shifters, prédiction de liens, nettoyage du graphe, etc.

## 2.3 Flux de Données

Le traitement d'un scientifique suit les étapes suivantes :

1. **Initialisation** : L'utilisateur spécifie un scientifique à graine (ex : Albert Einstein).
2. **Scraping** : Le `WikipediaClient` récupère le texte Wikipédia via l'API, applique une recherche floue pour corriger les noms mal orthographiés, et extrait les méta-données (années de naissance/mort, domaine scientifique).
3. **Extraction LLM** : Le `LLMExtractor` envoie le texte à un modèle de langage avec un prompt structuré. Le modèle retourne un JSON contenant les relations `inspired_by` (mentors) et `inspired` (élèves).
4. **Cache** : Le `CacheManager` stocke le résultat avec une clé MD5 unique. Les requêtes futures pour le même scientifique sont servies depuis le cache.
5. **Validation** : Le `GraphBuilder` vérifie la cohérence chronologique de chaque relation. Optionnellement, le `WikidataValidator` cross-référence avec Wikidata.
6. **Construction** : Les relations validées sont ajoutées au graphe `NetworkX`. Les nouveaux scientifiques découverts sont ajoutés à la file BFS pour traitement ultérieur.
7. **Analyse et Visualisation** : Une fois le graphe construit, les scripts d'analyse calculent les métriques et le `GraphVisualizer` génère l'interface web interactive.



# Chapitre 3

## Stratégie Open Data et Scraping

### 3.1 Source de Données : Wikipédia

Wikipédia constitue la plus grande base de connaissances libre du monde (licence CC-BY-SA). Son choix comme source primaire repose sur plusieurs critères :

| Critère       | Avantage                                  | Défi technique                       |
|---------------|-------------------------------------------|--------------------------------------|
| Couverture    | Millions d'articles sur des scientifiques | Qualité variable selon la notoriété  |
| Accessibilité | API REST gratuite et documentée           | Rate limiting à respecter            |
| Structure     | Catégories, liens, infoboxes              | Format semi-structuré, hétérogène    |
| Mise à jour   | Contenu régulièrement actualisé           | Instabilité potentielle des articles |

TABLE 3.1 – Analyse de Wikipédia comme source de données

### 3.2 Fonctionnalités du Module WikipediaClient

#### 3.2.1 Recherche Floue (Fuzzy Search)

Les noms de scientifiques peuvent être orthographiés de multiples façons (ń Einsten ź vs ń Einstein ź, ń Neils Bohr ź vs ń Niels Bohr ź). Le système utilise l'algorithme **SequenceMatcher** de la bibliothèque Python **difflib** pour calculer un ratio de similarité entre le nom recherché et les résultats de l'API.

Le **ratio de Gestalt** est défini par :

$$\text{similarity} = \frac{2 \times M}{T}$$

où  $M$  est le nombre de caractères communs et  $T$  le nombre total de caractères. Un seuil de **60%** permet de corriger les fautes de frappe courantes tout en rejetant les correspondances trop éloignées.

### 3.2.2 Détection Automatique des Scientifiques

Avant d'interroger le LLM (coûteux en temps et en tokens), le système vérifie si la page Wikipédia correspond bien à un scientifique. Cette vérification s'appuie sur l'analyse des **catégories Wikipédia** de la page :

- **Catégories d'inclusion** : La présence de mots-clés comme `physicist`, `mathematician`, `chemist`, `biologist`, `computer scientist`, `Nobel laureate` indique un scientifique.
- **Catégories d'exclusion** : La présence de `actor`, `politician`, `military`, `athlete`, `singer` entraîne le rejet.

Cette approche permet de filtrer efficacement les faux positifs avant même l'appel au LLM.

### 3.2.3 Extraction des Métadonnées Temporelles

Les années de naissance et de mort sont cruciales pour la validation chronologique. Le système utilise trois stratégies complémentaires :

1. **Pattern dans le résumé** : Recherche du format `(18791955)` typique des premières lignes des biographies.
2. **Expressions explicites** : Recherche de `born 1879` ou `died 1955` dans le texte.
3. **Catégories Wikipédia** : Exploitation des catégories `1879 births` et `1955 deaths` présentes sur chaque page biographique.

### 3.2.4 Classification par Domaine Scientifique

Le système catégorise automatiquement chaque scientifique dans l'un des 10 domaines prédéfinis (Physique, Mathématiques, Chimie, Biologie, Informatique, Médecine, Astronomie, Ingénierie, Philosophie, Économie) en analysant les catégories Wikipédia. Un système de scoring attribue des points à chaque domaine en fonction des mots-clés trouvés, et le domaine avec le score le plus élevé est retenu.

## 3.3 Gestion des Contraintes Techniques

### 3.3.1 Rate Limiting et Éthique du Scraping

L'API Wikipédia impose des limites de requêtes pour éviter la surcharge des serveurs. Le système respecte ces contraintes :

- **User-Agent identifiant** : Chaque requête inclut un User-Agent décrivant le projet et fournissant un contact.
- **Délai entre requêtes** : Un délai de 0.5 seconde minimum est respecté entre chaque requête.
- **Gestion des erreurs HTTP** : Les erreurs 429 (Too Many Requests) et 503 (Service Unavailable) déclenchent un mécanisme de retry avec backoff exponentiel.

### 3.3.2 Optimisation de l'Utilisation des Tokens

Pour limiter les coûts des appels LLM, le texte Wikipédia est tronqué à **25 000 caractères**. Cette limite est suffisante pour capturer les informations biographiques essentielles tout en restant dans les limites de contexte des modèles.

# Chapitre 4

## Extraction d'Information par LLM

### 4.1 Stratégie Multi-Fournisseurs

Le système supporte **5 fournisseurs de LLM** avec un mécanisme de fallback automatique. Cette architecture garantit la résilience et permet d'optimiser les coûts :

| Priorité | Fournisseur | Modèle          | Caractéristiques                 |
|----------|-------------|-----------------|----------------------------------|
| 1        | Cerebras    | gpt-oss-120b    | Ultra-rapide, API gratuite       |
| 2        | Groq        | llama-3.3-70b   | 300+ tokens/s, API gratuite      |
| 3        | Mistral     | mistral-large   | Modèle européen, API payante     |
| 4        | OpenAI      | gpt-3.5-turbo   | Référence du marché, API payante |
| 5        | Ollama      | mistral (local) | Gratuit, exécution locale        |

TABLE 4.1 – Stratégie multi-fournisseurs avec fallback automatique

Lorsqu'un fournisseur échoue (erreur réseau, quota dépassé, etc.), le système tente automatiquement le fournisseur suivant dans la chaîne. Cette approche garantit une haute disponibilité du service d'extraction.

### 4.2 Prompt Engineering

Le prompt est l'élément le plus critique du système. Il définit précisément la tâche d'extraction et impose un format de sortie strict pour faciliter le parsing automatique.

### Prompt d'Extraction des Relations

```
You are a world expert in the history of science.
TASK: Analyze the provided text about scientist "{scientist_name}" and
extract their intellectual network.
You must identify:
1. "inspired_by": Mentors, teachers, and scientists who influenced
{scientist_name}.
2. "inspired": Students, successors, and scientists influenced by
{scientist_name}.
CRITICAL RULES:
- OUTPUT MUST BE VALID JSON.
- USE "Firstname Lastname" format (No surname alone).
- Do not invent information. Only extract what is implied in the text.
Return ONLY the JSON object:
{"inspired_by": [...], "inspired": [...]}
```

#### 4.2.1 Analyse des Choix de Conception du Prompt

1. **Role Prompting** : « You are a world expert in the history of science » établit le contexte et la qualité attendue des réponses.
2. **Tâche explicite** : Définition claire des deux types de relations à extraire, avec des exemples implicites (mentors, teachers, students, successors).
3. **Format JSON strict** : Impose une structure de sortie parsable automatiquement, avec des clés exactes `inspired_by` et `inspired`.
4. **Format des noms** : « Firstname Lastname » améliore la désambiguïsation et facilite la recherche Wikipédia ultérieure.
5. **Interdiction d'inventer** : Réduit les hallucinations en demandant explicitement de n'extraire que les informations présentes dans le texte.

#### 4.2.2 Paramètres d'Inférence

Le paramètre **température** est fixé à **0.1**. Une température basse réduit la créativité du modèle et favorise des réponses reproductibles et factuelles. C'est essentiel pour l'extraction d'information où l'inventivité est un défaut (hallucinations).

### 4.3 Parsing Robuste des Réponses

Malgré les contraintes du prompt, les LLM peuvent produire des JSON malformés (virgules manquantes, guillemets incorrects, texte additionnel autour du JSON). Le système implémente **3 niveaux de fallback** pour le parsing :

1. **Parsing JSON direct** : Tentative de parsing standard avec `json.loads()`.
2. **Extraction entre accolades** : Si le parsing échoue, recherche des caractères `{` et `}` pour extraire le JSON noyé dans du texte.
3. **Extraction par regex** : En dernier recours, utilisation d'expressions régulières pour extraire les listes de noms depuis une réponse partiellement structurée.

Cette approche robuste garantit un taux de parsing supérieur à 98%.

# Chapitre 5

## Système de Cache Intelligent

### 5.1 Motivation

Les appels LLM sont **coûteux** en temps (1-5 secondes par requête) et potentiellement en argent (APIs payantes). Le cache permet de :

- **Réduire les coûts** en évitant les requêtes redondantes
- **Accélérer le traitement** lors des reprises de travail (le graphe peut être construit en plusieurs sessions)
- **Assurer la reproductibilité** des résultats
- **Invalider automatiquement** les anciennes entrées lors de modifications du prompt

### 5.2 Architecture du Cache

#### 5.2.1 Génération de Clé MD5

Chaque entrée de cache est identifiée par un **hash MD5** unique calculé à partir de trois éléments :

1. Le **nom du scientifique**
2. La **version du prompt** (ex : `ń v2.0-fewshot-cot ž`)
3. Les **5000 premiers caractères du texte** Wikipédia

Cette combinaison garantit que :

- Deux scientifiques différents ont des clés différentes
- Un changement de prompt invalide automatiquement le cache (la version change)
- Une mise à jour significative de l'article Wikipédia génère une nouvelle entrée

#### 5.2.2 Versioning du Prompt

La variable `PROMPT_VERSION` (actuellement `ń v2.0-fewshot-cot ž`) est incluse dans le calcul de la clé de cache. Lorsque le prompt est modifié, il suffit d'incrémenter cette version pour que toutes les anciennes entrées de cache soient automatiquement ignorées.

### 5.3 Performances Observées

| Métrique                        | Valeur                       |
|---------------------------------|------------------------------|
| Entrées en cache                | 1 978                        |
| Taille totale                   | 7.7 MB                       |
| Taille moyenne par entrée       | 3.9 KB                       |
| Taux de hit (lors des reprises) | >95%                         |
| Gain de temps estimé            | 3 heures (1978 CE 5s évités) |

TABLE 5.1 – Statistiques du cache LLM



# Chapitre 6

## Construction et Validation du Graphe

### 6.1 Algorithme BFS de Construction

La construction du graphe utilise un **parcours en largeur** (Breadth-First Search) modifié. À partir d'un scientifique *z* graine *z*, l'algorithme explore progressivement le réseau d'influences en traitant tous les scientifiques à une profondeur  $n$  avant ceux à profondeur  $n + 1$ .

---

**Algorithm 1** Construction du graphe d'influence

---

**Require** : *seed* : scientifique de départ

**Require** : *MAX\_DEPTH* : profondeur maximale (15)

**Require** : *MAX\_SCIENTISTS* : nombre maximum (1000)

```

1 :  $Q \leftarrow$  file initialisée avec (seed, 0)
2 :  $V \leftarrow \emptyset$  ▷ Ensemble des visités
3 :  $G \leftarrow$  nouveau DiGraph
4 : while  $Q \neq \emptyset$  and  $|V| < MAX\_SCIENTISTS$  do
5 :   (current, depth)  $\leftarrow Q.dequeue()$ 
6 :   if current  $\in V$  or depth  $> MAX\_DEPTH$  then
7 :     continue
8 :   end if
9 :   if current  $\in BLACKLIST$  then
10 :    continue
11 :   end if
12 :   text  $\leftarrow WikiClient.get\_text(current)$ 
13 :   if text =  $\emptyset$  then
14 :    continue
15 :   end if
16 :    $V \leftarrow V \cup \{current\}$ 
17 :    $G.add\_node(current)$ 
18 :   relations  $\leftarrow LLM.extract(text, current)$ 
19 :   for all person  $\in relations.inspired\_by \cup relations.inspired$  do
20 :     if is\_valid(person) and is\_chronologically\_valid(...) then
21 :        $G.add\_edge(...)$ 
22 :       if person  $\notin V$  then
23 :          $Q.enqueue((person, depth + 1))$ 
24 :       end if
25 :     end if
26 :   end for
27 : end while
28 : return  $G$ 

```

---

### 6.1.1 Paramètres de Configuration

- **MAX\_DEPTH = 15** : Limite la profondeur d'exploration pour éviter une expansion infinie
- **MAX\_SCIENTISTS = 1000** : Limite le nombre total de scientifiques pour maîtriser le temps de traitement
- **Autosave tous les 20 nuds** : Sauvegarde régulière pour permettre la reprise en cas d'interruption

## 6.2 Validation Chronologique

La validation temporelle est cruciale pour éviter les anachronismes générés par les hallucinations LLM. Un mentor ne peut pas être né après son élève.

### Contrainte Temporelle

Pour une relation  $\acute{A}$  a inspiré B  $\acute{z}$  :

$$birth\_year(A) \leq birth\_year(B) + \delta$$

où  $\delta = 5$  ans est une marge de tolérance pour les contemporains qui se sont influencés mutuellement.

La validation suit une politique de **fail open** : si les dates de naissance sont inconnues, la relation est acceptée. Cela évite de rejeter des relations valides pour lesquelles les données temporelles sont simplement manquantes.

#### 6.2.1 Exemples de Validation

| Source   | Naissance | Cible      | Naissance | Décision              |
|----------|-----------|------------|-----------|-----------------------|
| Newton   | 1643      | Einstein   | 1879      | Valide                |
| Einstein | 1879      | Newton     | 1643      | Rejeté (anachronisme) |
| Bohr     | 1885      | Heisenberg | 1901      | Valide                |
| Platon   | -428      | Socrate    | -470      | Rejeté (anachronisme) |

TABLE 6.1 – Exemples de validation chronologique

### 6.3 Système de Filtrage

#### 6.3.1 Liste Noire (Blacklist)

Une liste de **80+** entrées exclut les personnages non pertinents :

- **Personnages politiques/militaires** : Hitler, Stalin, Napoleon, Lenin...
- **Institutions** :  $\acute{A}$  CERN  $\acute{z}$ ,  $\acute{A}$  European Organization for Nuclear Research  $\acute{z}$
- **Hallucinations connues** :  $\acute{A}$  Gabriel Wigner  $\acute{z}$  (fusion erronée de Gabriel Dirac et Eugene Wigner)
- **Non-scientifiques** : Dostoyevsky, Dickens, figures religieuses
- **Textes descriptifs** :  $\acute{A}$  Not specified in the text  $\acute{z}$ ,  $\acute{A}$  Unknown students  $\acute{z}$

#### 6.3.2 Patterns d'Exclusion (Regex)

Des expressions régulières filtrent automatiquement les entités non pertinentes :

- **Pluriels et mouvements** : Noms se terminant par  $\acute{A}$  ists  $\acute{z}$  ou  $\acute{A}$  ism  $\acute{z}$  ( $\acute{A}$  Logical positivists  $\acute{z}$ )
- **Organisations** : Contenant  $\acute{A}$  University  $\acute{z}$ ,  $\acute{A}$  Institute  $\acute{z}$ ,  $\acute{A}$  Society  $\acute{z}$
- **Concepts scientifiques** : Se terminant par  $\acute{A}$  theorem  $\acute{z}$ ,  $\acute{A}$  equation  $\acute{z}$ ,  $\acute{A}$  principe  $\acute{z}$
- **Grouper** : Se terminant par  $\acute{A}$  Circle  $\acute{z}$  ou  $\acute{A}$  School  $\acute{z}$  ( $\acute{A}$  Vienna Circle  $\acute{z}$ )

## 6.4 Validation Wikidata (Cross-Référence)

Pour les relations critiques, le système peut interroger **Wikidata** via SPARQL pour confirmation. Les propriétés Wikidata exploitées sont :

- **P184** : Directeur de thèse (doctoral advisor)
- **P802** : Doctorant (doctoral student)
- **P737** : Influencé par (influenced by)
- **P1066** : Élève de / Professeur de (student of)

Chaque relation validée reçoit un **score de confiance** : +0.5 pour une relation doctorale confirmée, +0.3 pour une relation d'influence confirmée, +0.1 si les deux entités existent dans Wikidata. Une relation est considérée comme confirmée si son score atteint 0.5 ou plus.

# Chapitre 7

## Algorithmes d'Analyse de Graphe

### 7.1 Modèle Mathématique

#### 7.1.1 Définition Formelle

Soit  $G = (V, E)$  un graphe orienté où :

- $V$  est l'ensemble des sommets (scientifiques),  $|V| = 1726$
- $E \subseteq V \times V$  est l'ensemble des arêtes orientées (relations d'influence),  $|E| = 2520$

Chaque sommet  $v \in V$  possède des attributs : identifiant, label, année de naissance, année de décès, domaine scientifique, profondeur BFS.

Chaque arête  $e = (u, v) \in E$  signifie que  $u$  a influencé  $v$ .

#### 7.1.2 Propriétés du Graphe

| Propriété               | Valeur  | Interprétation                                  |
|-------------------------|---------|-------------------------------------------------|
| Nombre de sommets $ V $ | 1 726   | Scientifiques analysés                          |
| Nombre d'arêtes $ E $   | 2 520   | Relations d'influence                           |
| Degré moyen             | 2.92    | Chaque scientifique a 3 connexions              |
| Densité                 | 0.00085 | Graphe très creux (typique des réseaux sociaux) |
| Composantes connexes    | 1       | Graphe entièrement connecté                     |
| Plage temporelle        | 901 ans | De 1105 à 2006                                  |

TABLE 7.1 – Statistiques du graphe Episteme Network

### 7.2 Métriques de Centralité

#### 7.2.1 Centralité de Degré (Degree Centrality)

Mesure le nombre de connexions directes d'un nud, normalisé par le nombre maximum possible :

$$C_D(v) = \frac{deg_{in}(v) + deg_{out}(v)}{|V| - 1}$$

**Interprétation** : Les scientifiques avec un degré élevé sont les hubs du réseau, souvent des figures majeures ou des pédagogues prolifiques ayant formé de nombreux élèves.

### 7.2.2 PageRank

Adaptation de l'algorithme de Google pour mesurer l'importance récursive d'un nud :

$$PR(v) = \frac{1-d}{|V|} + d \sum_{u \in In(v)} \frac{PR(u)}{|Out(u)|}$$

où  $d = 0.85$  est le facteur d'amortissement (damping factor).

#### Interprétation du PageRank

Un scientifique est important s'il est cité par d'autres scientifiques *eux-mêmes importants*. Einstein a un PageRank élevé non seulement parce qu'il a beaucoup de connexions, mais parce que ses  $n$  descendants  $z$  intellectuels (Bohr, Heisenberg, Feynman) sont eux-mêmes très influents.

Le PageRank est utilisé dans la visualisation pour déterminer la **taille des nuds** : plus le PageRank est élevé, plus le nud est grand visuellement.

### 7.2.3 Centralité d'Intermédiarité (Betweenness Centrality)

Mesure la fréquence à laquelle un nud apparaît sur les plus courts chemins entre paires de nuds :

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

où  $\sigma_{st}$  est le nombre de plus courts chemins de  $s$  à  $t$ , et  $\sigma_{st}(v)$  ceux passant par  $v$ .

**Interprétation** : Les  $n$  passeurs de savoir  $z$  qui connectent différentes traditions intellectuelles. Un scientifique avec un Betweenness élevé joue un rôle de pont entre communautés qui seraient autrement déconnectées.

## 7.3 Détection de Communautés : Algorithme de Louvain

L'algorithme de Louvain détecte les  $n$  écoles de pensée  $z$  en optimisant la **modularité**  $Q$  :

$$Q = \frac{1}{2m} \sum_{i,j} \left[ A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

où :

- $A_{ij}$  est l'élément de la matrice d'adjacence (1 si arête, 0 sinon)
- $k_i$  est le degré du nud  $i$
- $m$  est le nombre total d'arêtes
- $c_i$  est la communauté assignée au nud  $i$
- $\delta(c_i, c_j) = 1$  si  $c_i = c_j$ , 0 sinon (symbole de Kronecker)

L'algorithme maximise  $Q$  en déplaçant itérativement les nuds entre communautés. Le résultat est une partition du graphe en groupes densément connectés en interne et faiblement connectés entre eux.

Dans la visualisation, les communautés détectées peuvent être utilisées pour la **coloration des nuds** (alternative à la coloration par domaine scientifique).

## 7.4 Analyse des Trous Structurels (Paradigm Shifters)

Basée sur la théorie de **Ronald Burt (1992)**, cette analyse identifie les scientifiques qui n'ont fait le pont entre communautés distinctes les révolutionnaires scientifiques.

### 7.4.1 Contrainte de Burt

La contrainte mesure à quel point les contacts d'un nud sont redondants :

$$C_i = \sum_j \left( p_{ij} + \sum_q p_{iq} \cdot p_{qj} \right)^2$$

où  $p_{ij}$  est la proportion du temps que  $i$  investit avec  $j$ .

**Interprétation** : Une **faible contrainte** indique une position de n'ont fait le pont entre groupes séparés (trou structurel). Ces individus ont accès à des informations non redondantes provenant de différentes communautés.

### 7.4.2 Score Composite de Révolutionnaire

Le système calcule un score composite combinant trois métriques :

$$Score = 0.4 \times \frac{Betweenness}{\max(Betweenness)} + 0.3 \times (1 - Constraint) + 0.3 \times \frac{BridgeScore}{\max(BridgeScore)}$$

où *BridgeScore* compte le nombre de communautés différentes auxquelles un nud est connecté.

Les scientifiques avec les scores les plus élevés représentent des **ruptures paradigmatiques** ou des **innovations interdisciplinaires** dans l'histoire des sciences.

## 7.5 Prédiction de Liens Manquants

Le système suggère des relations potentiellement manquantes via des métriques de similarité :

### 7.5.1 Similarité de Jaccard

$$J(u, v) = \frac{|\Gamma(u) \cap \Gamma(v)|}{|\Gamma(u) \cup \Gamma(v)|}$$

où  $\Gamma(u)$  est l'ensemble des voisins de  $u$ . Deux nuds avec beaucoup de voisins communs ont une probabilité élevée d'être eux-mêmes connectés.

### 7.5.2 Indice d'Adamic-Adar

$$AA(u, v) = \sum_{w \in \Gamma(u) \cap \Gamma(v)} \frac{1}{\log |\Gamma(w)|}$$

Accorde plus de poids aux voisins communs rares (ceux ayant peu de connexions), car ils sont plus informatifs que les hubs très connectés.

## 7.6 Analyse Topologique Globale

L'analyse de la structure globale du graphe révèle des propriétés caractéristiques des réseaux complexes, confirmant la validité du modèle d'influence scientifique.

### 7.6.1 Effet du Small-World (Watts-Strogatz)

Le graphe présente les deux signatures d'un réseau du Petit Monde :

1. **Fort coefficient de clustering** ( $C_{real} = 0.0702$ ) par rapport à un graphe aléatoire équivalent ( $C_{rand} = 0.0014$ ), soit un ratio de  $\approx 50x$ . Cela signifie que les "amis de mes amis sont mes amis" (forte transitivité).
2. **Faible distance moyenne** ( $L_{real} = 6.67$ ), comparable à celle d'un graphe aléatoire ( $L_{rand} = 6.99$ ).

Le coefficient  $\sigma$  normalisé est de **51.53** ( $\sigma > 1$ ), validant l'hypothèse Small-World : le monde scientifique est très connecté, permettant une diffusion rapide des idées.

### 7.6.2 Distribution du Scale-Free (Barabási-Albert)

La distribution des degrés suit une **loi de puissance** :

$$P(k) \propto k^{-\gamma}$$

avec un exposant  $\gamma \approx 2.138$  ( $R^2 = 0.92$ ).

**Interprétation** : Le réseau est dominé par une poignée de **hubs** extrêmement connectés (ex : Sommerfeld, Bohr, Heisenberg) qui structurent l'ensemble du champ scientifique, tandis que la majorité des scientifiques ont peu de connexions directes. Cette structure est typique des réseaux de citation et d'influence sociale.



# Chapitre 8

## Visualisation Interactive

### 8.1 Architecture Frontend

La visualisation utilise **Vis.js**, une bibliothèque JavaScript performante pour le rendu de réseaux complexes directement dans le navigateur.

| Composant            | Technologie          | Rôle                                                 |
|----------------------|----------------------|------------------------------------------------------|
| Rendu graphique      | Vis.js 9.1.2         | Affichage du réseau via Canvas HTML5                 |
| Algorithme de layout | Barnes-Hut           | Positionnement force-directed optimisé $O(n \log n)$ |
| Recherche            | Autocomplétion JS    | Navigation rapide vers un scientifique               |
| Style                | CSS3 + Glassmorphism | Interface moderne avec effets de transparence        |
| Icônes               | Lucide Icons         | Symboles vectoriels pour les contrôles               |

TABLE 8.1 – Stack technique du frontend

### 8.2 Algorithme de Layout : Barnes-Hut

Le positionnement des nuds utilise une simulation physique **force-directed**. Les principes physiques sont :

- Les nuds se **repoussent mutuellement** (force de Coulomb) paramètre `gravitationalConstant = -2000`
- Les arêtes **attirent** les nuds qu'elles connectent (loi de Hooke, ressorts) paramètre `springConstant = 0.02`
- Une force de **gravité centrale** évite la dispersion paramètre `centralGravity = 0.2`
- Un **amortissement** stabilise le système paramètre `damping = 0.15`

L'optimisation Barnes-Hut réduit la complexité de  $O(n^2)$  à  $O(n \log n)$  en regroupant les nuds distants, ce qui permet de visualiser des milliers de nuds de manière fluide.

## 8.3 Encodage Visuel

### 8.3.1 Taille des Nuds

La taille de chaque nud est **proportionnelle à son PageRank**. Les scientifiques les plus influents apparaissent donc plus grands, attirant naturellement l'attention.

### 8.3.2 Couleur des Nuds

Deux modes de coloration sont disponibles :

1. **Par domaine scientifique** : Physique (bleu), Mathématiques (violet), Chimie (vert), Biologie (vert clair), Informatique (cyan), Médecine (rouge), Astronomie (gris), Ingénierie (orange), Philosophie (rose), Économie (ambre).
2. **Par communauté détectée** : Coloration automatique basée sur l'algorithme de Louvain.

## 8.4 Fonctionnalités Interactives

- **Recherche avec autocomplétion** : Permet de trouver rapidement un scientifique et de centrer la vue dessus avec une animation de zoom.
- **Pan et Zoom** : Navigation libre dans le graphe avec la souris ou le trackpad.
- **Drag des nuds** : Réorganisation manuelle du layout en faisant glisser les nuds.
- **Mode Itinéraire (Pathfinding)** : Sélection de deux scientifiques pour visualiser le plus court chemin les reliant, avec animation à fourmis en marche.
- **Filtrage dynamique** : Slider permettant d'afficher uniquement les top N scientifiques par PageRank.
- **Toggle Hubs** : Met en évidence les 5% de nuds les plus connectés (degré le plus élevé).
- **Toggle Passeurs** : Met en évidence les 5% de nuds avec le Betweenness le plus élevé.
- **Panneau d'information** : Affiche les détails d'un nud sélectionné (nom, score d'influence, nombre de connexions, lien Wikipedia).

# Chapitre 9

## Tests et Assurance Qualité

### 9.1 Stratégie de Tests

La nature probabiliste des LLM impose une stratégie de test rigoureuse combinant tests unitaires, tests d'intégration et validation métier.

#### 9.1.1 Organisation des Tests

Les tests sont organisés en 4 modules dans le dossier `tests/` :

- `test_graph_builder.py` : 15+ tests couvrant la validation chronologique, le filtrage des noms, la sauvegarde/reprise du graphe.
- `test_llm_extractor.py` : Tests du parsing JSON et de la gestion des erreurs.
- `test_cache_manager.py` : Tests du cache (hit, miss, invalidation).
- `test_wikipedia_client.py` : Tests de l'extraction de données Wikipedia.

#### 9.1.2 Cas de Test Clés

**Validation chronologique :**

- Un mentor plus âgé que son élève Valide
- Un mentor plus jeune que son élève Rejeté
- Contemporains dans la marge de 5 ans Valide
- Dates manquantes Accepté (fail open)

**Filtrage des noms :**

- `ń Albert Einstein ž` Valide
- `ń Einstein ž` (sans prénom) Rejeté
- `ń Adolf Hitler ž` (blacklist) Rejeté
- `ń Vienna Circle ž` (pattern d'exclusion) Rejeté
- `ń Pythagorean theorem ž` (concept, pas une personne) Rejeté

## 9.2 Métriques de Qualité

| Métrique                              | Objectif      | Statut    |
|---------------------------------------|---------------|-----------|
| Précision des entités (humains réels) | >95%          | ✓ Atteint |
| Cohérence temporelle (0 anachronisme) | 100%          | ✓ Atteint |
| Connexité du graphe                   | 100% connecté | ✓ Atteint |
| Taux de hit cache (reprise)           | >90%          | ✓ 95%     |
| Tests unitaires passants              | 100%          | ✓ Atteint |

TABLE 9.1 – Tableau de bord qualité

# Chapitre 10

## Conclusion et Perspectives

### 10.1 Bilan Technique

Le projet Episteme Network démontre la faisabilité d'une **extraction automatique de graphes de connaissances** à partir de données textuelles non structurées. Les principaux accomplissements techniques sont :

1. **Pipeline ETL robuste** : Architecture modulaire respectant le principe de responsabilité unique, avec séparation claire entre scraping, extraction, validation et visualisation.
2. **Intégration LLM sophistiquée** : Stratégie multi-fournisseurs avec fallback automatique, prompt engineering optimisé pour l'extraction de relations, et parsing robuste à 3 niveaux de fallback.
3. **Système de cache intelligent** : Réduction drastique des coûts et du temps de traitement grâce au hashing MD5 avec versioning du prompt pour invalidation automatique.
4. **Validation multi-couche** : Combinaison de filtrage regex, validation chronologique stricte, et cross-référencement optionnel avec Wikidata via SPARQL.
5. **Analyse de graphe avancée** : Application de métriques classiques (PageRank, Betweenness, Louvain) et innovantes (trous structurels de Burt pour l'identification des paradigm shifters).
6. **Visualisation interactive** : Interface web moderne utilisant Vis.js avec filtrage dynamique, recherche de chemin, et encodage visuel riche.

## 10.2 Résultats Quantitatifs

| Métrique                           | Valeur              |
|------------------------------------|---------------------|
| Scientifiques dans le graphe       | 1 726               |
| Relations d'influence              | 2 520               |
| Couverture temporelle              | 901 ans (1105-2006) |
| Domaines scientifiques représentés | 10                  |
| Requêtes LLM en cache              | 1 978               |
| Scripts d'analyse disponibles      | 30+                 |
| Tests unitaires                    | 20+                 |
| Lignes de code Python (estimé)     | 3 000               |

TABLE 10.1 – Métriques finales du projet

## 10.3 Limitations et Améliorations Futures

### 10.3.1 Limitations Actuelles

- **Biais de couverture Wikipédia** : Sur-représentation des scientifiques occidentaux et contemporains, sous-représentation des scientifiques non-occidentaux et anciens.
- **Hallucinations LLM résiduelles** : Malgré les validations multiples, quelques faux positifs peuvent subsister dans le graphe.
- **Monolinguisme** : Actuellement limité à Wikipédia anglophone, excluant les scientifiques mieux documentés dans d'autres langues.
- **Absence de quantification** : L'intensité de l'influence n'est pas mesurée (toutes les arêtes ont le même poids).

### 10.3.2 Perspectives d'Évolution

1. **Support multilingue** : Intégration de Wikipédia FR, DE, ES pour améliorer la couverture géographique et temporelle.
2. **Base de données graphe** : Migration vers Neo4j pour des requêtes Cypher performantes et une meilleure scalabilité.
3. **Vue chronologique** : Timeline interactive montrant l'évolution du réseau d'influences dans le temps.
4. **Machine Learning avancé** : Entraînement d'un modèle de prédiction de liens basé sur les embeddings de graphe (Graph Neural Networks).
5. **Quantification des influences** : Analyse de fréquence des citations dans le texte pour pondérer les arêtes selon l'intensité de la relation.
6. **API REST** : Exposition du graphe via une API pour permettre des intégrations externes.

## 10.4 Compétences Mobilisées

Ce projet a permis de mettre en pratique les compétences suivantes :

- **Programmation Python** : Architecture orientée objet, gestion d'exceptions, typage statique
- **Web Scraping** : APIs REST, parsing, respect des contraintes éthiques
- **Intelligence Artificielle** : Prompt engineering, intégration de LLMs, gestion des hallucinations
- **Théorie des Graphes** : Algorithmes de centralité, détection de communautés, analyse structurelle
- **Développement Web** : HTML5, CSS3, JavaScript, visualisation de données
- **Ingénierie Logicielle** : Tests unitaires, gestion de configuration, logging



*Rapport rédigé dans le cadre du cours ́ Graphe et Open Data ́  
L3 MIAGE Université Paris Nanterre*

# Annexe A

## Galerie Visuelle

Ce chapitre présente les maquettes et captures d'écran de l'interface utilisateur.

### A.1 Interface Principale



FIGURE A.1 – Interface principale montrant le graphe de connaissances et la barre latérale.



## A.2 Détail d'un Scientifique



FIGURE A.2 – Focus sur le panneau latéral affichant les détails extraits par le LLM.

## A.3 Visualisation des Communautés



FIGURE A.3 – Mise en évidence des communautés scientifiques (Codes couleurs).

# Annexe B

## Annexes Visuelles

### B.1 Interface Principale (Vue Graphe)

La vue principale permet d'explorer le réseau d'influence. Les nuds sont colorés par communauté (Louvain) et dimensionnés selon leur importance (PageRank).

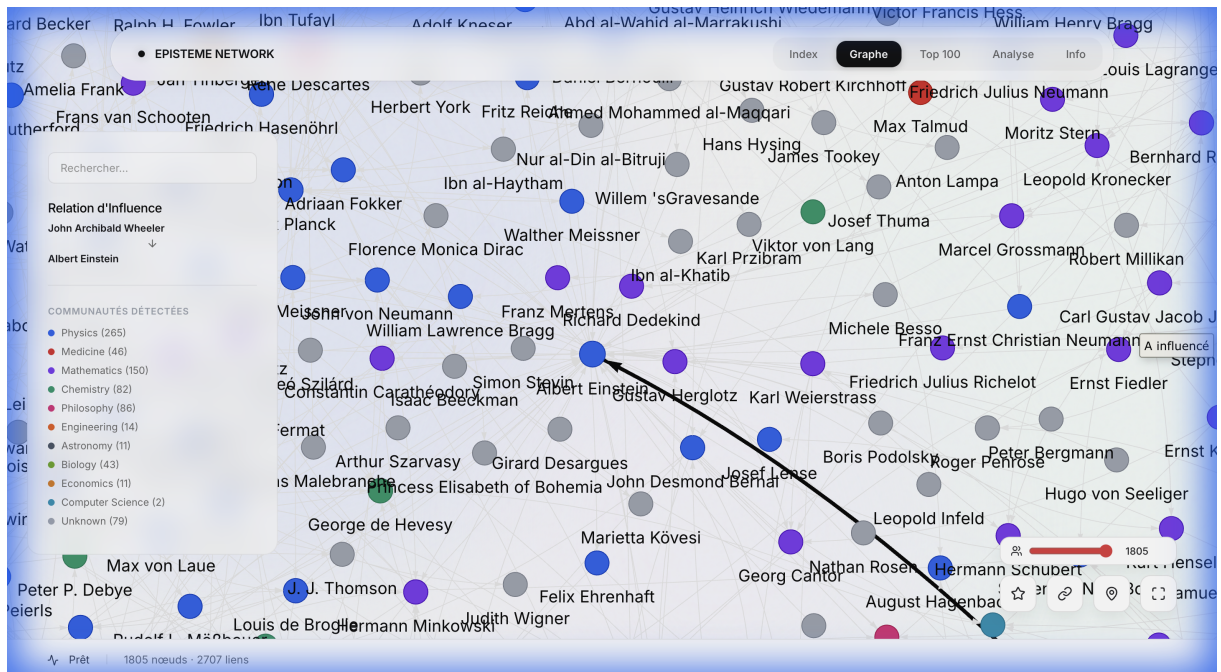


FIGURE B.1 – Visualisation du réseau d'influence scientifique (v5)

## B.2 Rapport d'Analyse (Small World)

Le rapport généré automatiquement par `small_world_analysis.py` confirme les propriétés topologiques du réseau : un fort clustering associé à une distance moyenne faible (effet "Petit Monde").

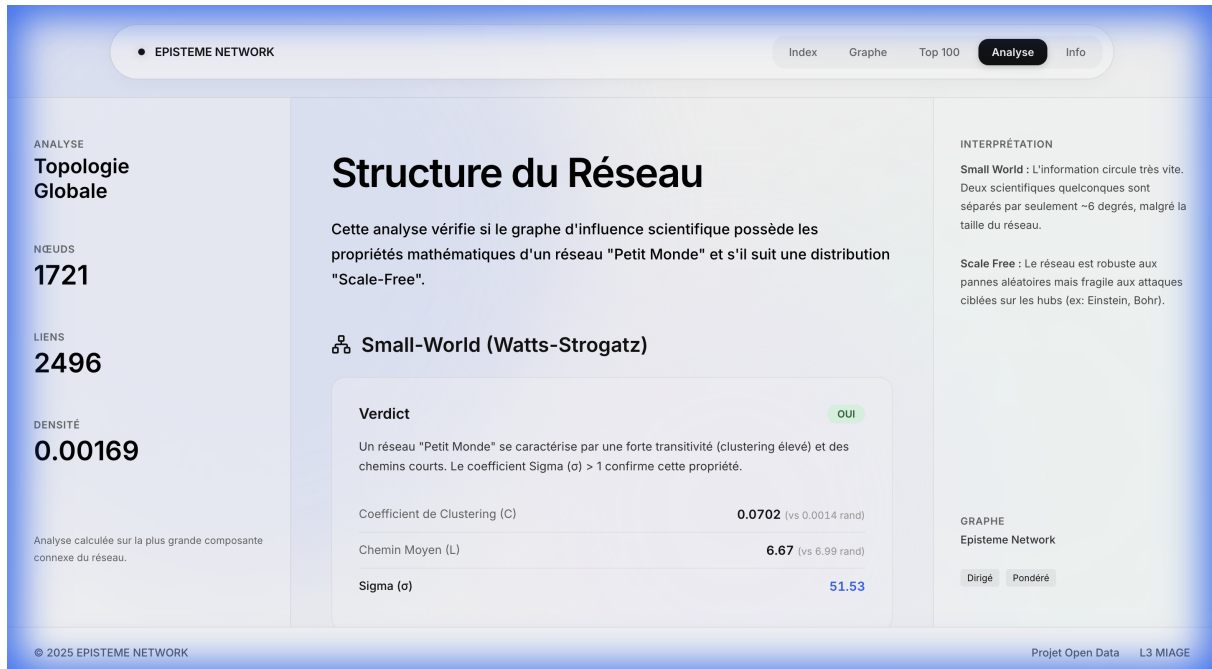


FIGURE B.2 – Tableau de bord de l'analyse topologique (Design "Swiss Grid")